

i.MX6ULL LED 汇编点灯

——基于 NXP i.MX6ULL 处理器的 GPIO 汇编控制

一、开发环境

- 硬件平台：NXP i.MX6ULL（IMX Mini / IMX 阿尔法）
- DDR 起始地址：0x8000_0000，容量 32M（0x8000_0000 ~ 0x8400_0000）
- 代码在 Ubuntu 上交叉编译，通过 VSCode 编辑。

VSCode 所需扩展：

- ☐ Arm Assembly — ARM 汇编语言支持
- ☐ Makefile Tools — Makefile 构建工具
- ☐ C/C++ Extension Pack — C/C++ 开发扩展包
- ☐ Chinese (Simplified) — 简体中文界面汉化

二、内存地址与栈设置

i.MX6ULL 启动后，DDR 映射在物理地址 0x8000_0000。汇编点灯需要设置栈指针，以便在异常或函数调用时有合法的堆栈空间。

寄存器/地址	含义	典型值
sp_irq	IRQ 模式栈指针	0xA000_0000
sp_system	System 模式栈指针	0x8600_0000
DDR 基地址	DDR 起始地址	0x8000_0000

三、点灯三种方式

i.MX6ULL 控制 LED 有三种常见实现路径，从底层到高层逐步封装：

方式一：纯汇编点灯

直接通过汇编指令读写寄存器，配置 IOMUXC、GPIO、CCM 三个模块，实现 LED 亮灭。适合理解硬件启动过程，无需 C 运行时。

方式二：C 语言点灯

在汇编初始化栈、时钟、GPIO 后，跳转到 C 函数中执行逻辑控制。C 代码可读性更好，但需要保证汇编启动代码正确配置运行环境。

方式三：NXP 固件库点灯

调用 NXP 官方提供的固件库（如 fsl_gpio.h），通过封装好的 API 配置 GPIO，代码简洁，开发效率高，但屏蔽了底层寄存器细节。

四、控制 LED 的三大核心模块

无论哪种方式点灯，最终都要配置以下三个硬件模块：

1. IOMUXC（引脚复用控制器）

负责将芯片引脚配置为特定功能（如 GPIO、UART、I2C 等）。点灯需配置两个寄存器：

- ☐ IOMUXC_SW_MUX_CTL_PAD_XXX：选择引脚复用功能（如 ALT5 = GPIO）
- ☐ IOMUXC_SW_PAD_CTL_PAD_XXX：配置电气属性（驱动能力、上下拉、速度等）

关键逻辑：先复用，再设电气属性。复用功能选 GPIO 模式后，引脚才会受 GPIO 模块控制。

2. GPIO（通用输入输出）

控制引脚输入/输出状态。点灯需操作两个核心寄存器：

寄存器名	全称	作用
GPIO1_DR	Data Register	数据寄存器：写 1 亮灯，写 0 灭灯（输出模式）
GPIO1_GDIR	Direction Register	方向寄存器：配置引脚为输入（0）或输出（1）

操作顺序：先设 GDIR 为输出模式（bit 置 1），再写 DR 控制电平。

3. CCM（时钟控制模块）

负责外设时钟门控。i.MX6ULL 的外设默认时钟可能关闭，需要手动开启。

- ☐ CCM_CCGR0 ~ CCM_CCGR6：共 7 个时钟门控寄存器，每个寄存器控制多个外设的时钟使能。
- ☐ 快捷配置：全部写 0xFFFF_FFFF，一次性打开所有外设时钟。实际项目中建议按需开启，降低功耗。

五、开发流程

VSCode 编写代码 → 保存至 Ubuntu 虚拟机共享目录 → 交叉编译（arm-linux-gnueabi-gcc/as/ld）→ 生成 .bin 文件 → 烧录到 i.MX6ULL 开发板 → 观察 LED 状态。

六、关键点

- ☐ 点灯之前必须配置 IOMUXC（复用功能），否则 GPIO 模块无法控制该引脚。
- ☐ GPIO 方向寄存器 GDIR 必须先配置为输出模式，再写数据寄存器 DR。
- ☐ CCM 时钟必须打开，否则 GPIO 模块不工作，即使寄存器写对了灯也不亮。
- ☐ 栈指针 SP 在启动阶段必须初始化，否则进入异常模式或函数调用会崩溃。
- ☐ DDR 基地址 0x8000_0000 是 i.MX6ULL 的内存映射起点，所有代码/数据链接地址需与此匹配。